(below): boxplots for the distribution of running times for each branching factor. All charts are created using data about the running time per single call, for the compression of an ensemble of text files.

Before delving into the results, though, we need to take a step back: since we will use a boxplot to display our data, let's make sure we know how to interpret these plots first: if you feel you could use a hint, figure 2.19 comes to the rescue!

To make sense of the data our example notebook uses *Pandas* library, with which we read the CSV file with the stats and transform it into a `DataFrame`, an internal Pandas representation - you can think about it as a SQL table on steroids: in fact, using this `DataFrame` we can partition data by test case (*image* versus *text*), then group it by method, and finally process each method separately and display results, for each method, grouped by branching factor. Figure 2.20 shows these results for encoding a large text, comparing the two main functions in Huffman encoding that use a heap.

If we look at those results, we can confirm a few points we mentioned in section 2.9:

- 2 is never the best choice for a branching factor;
- A branching factor of 4 or 5 seems a good compromise;
- There is a consistent difference (even -50% for `_frequency_table_to_heap`) between a binary heap and the d-ary heap with best branching factor.